

OS Lab 2 Report

Processes, Scheduling and Concurrent File Access

Efe Kardes - 2757072
Emir Avni Sahin - 2756541

June 2, 2026

1 Lab 1: GPIO Toggling on Raspberry Pi

In the first lab, GPIO17 of the Raspberry Pi was toggled and measured. GPIO17 corresponds to physical pin 11 on the Raspberry Pi header. The goal was to compare two methods: toggling the pin with the Linux GPIO tool `gpio`, and toggling the pin with direct register access in C.

1.1 Experimental Setup

The Raspberry Pi was connected to the lab monitor and accessed through the Linux terminal. Jumper wires were connected to the GPIO header. GPIO17 was used as the output signal, and a ground pin was used as the reference for the measurement.

1.2 Shell Script Method

The first method used the Linux GPIO character device interface through the `gpio` command. The important part of the shell script was:

```
while true
do
    gpio -c gpiochip0 -p 500ms -t0 17=1
    gpio -c gpiochip0 -p 500ms -t0 17=0
done
```

This repeatedly sets GPIO17 to high and then to low. This method is simple, but it is slow because every change is done through a user-space command and the Linux GPIO interface.

A faster `gpio` version can use the `toggle` option directly:

```
gpio -c gpiochip0 -t 1us,1us 17=1
```

This avoids starting a new `gpio` command for every single high and low state.



Figure 1: Raspberry Pi GPIO17 measurement setup.

1.3 Direct Register Access Method

The second method used a C program with direct access to the GPIO registers. The important idea is to map the GPIO register area into the process address space and then write directly to the set and clear registers.

The important part of the C approach is:

```
int fd = open("/dev/mem", O_RDWR | O_SYNC);

gpio = mmap(NULL, BLOCK_SIZE,
            PROT_READ | PROT_WRITE,
            MAP_SHARED,
            fd,
            GPIO_BASE);

while (1) {
    gpio[GPIO_SET_OFFSET] = (1 << 17);
    gpio[GPIO_CLR_OFFSET] = (1 << 17);
}
```

```
}
```

The call to `open()` gives access to physical memory, and `mmap()` maps the GPIO register area into the program. After that, the program can set and clear GPIO17 by writing directly to the GPIO set and clear registers.

1.4 Result

The shell-script method is easier to write and understand, but it has high overhead because it uses command-line tools and the kernel GPIO interface. The direct register-access method is more complex, but it is much faster because it avoids repeated command execution and writes directly to the hardware registers.

Therefore, the expected result is that the direct C implementation reaches a much higher toggle frequency than the shell-script method.

2 Lab 2 - Introduction

The goal of this lab was to observe how Linux handles processes, priorities and concurrent access to shared resources. The lab had two main parts. In the first part, two processes competed for CPU time while writing characters to the terminal. In the second part, two processes accessed the same file in a fixed order, first using a lock file and then using `fcntl()` locking.

The required tasks were based on the OS Lab 2 assignment, which focuses on process priorities, scheduling and locking of concurrent file access. The assignment requires two competing processes for scheduling and two processes A and B that alternately write, read and erase a shared file [1].

3 Task 2.1: Scheduling

3.1 Program Idea

For the scheduling task, I used a C program called `scheduling.c`. The program creates two child processes with `fork()`.

- Process A prints the character A.
- Process B prints the character B.

The parent process only creates the two child processes, prints their process IDs, waits for the selected time and then stops both children.

The basic structure is:

```
Parent process
|-- Child A prints A
|-- Child B prints B
```

The important code part is:

```
pid_t child_a = fork();

if (child_a == 0) {
    child_loop('A', method, sleep_us);
}

pid_t child_b = fork();
```

```

if (child_b == 0) {
    child_loop('B', method, sleep_us);
}

```

The function `fork()` creates a new process. In the child process, `fork()` returns 0. In the parent process, it returns the PID of the child.

3.2 The `child_loop()` Function

The child processes run the function `child_loop()`:

```

static void child_loop(char character, const char *method, useconds_t
    sleep_us)
{
    signal(SIGTERM, handle_signal);
    signal(SIGINT, handle_signal);

    if (strcmp(method, "printf") == 0) {
        setvbuf(stdout, NULL, _IOFBF, 4096);
    } else if (strcmp(method, "printf-flush") == 0) {
        setvbuf(stdout, NULL, _IONBF, 0);
    }

    while (!stop_requested) {
        if (strcmp(method, "write") == 0) {
            write(STDOUT_FILENO, &character, 1);
        } else if (strcmp(method, "printf") == 0) {
            printf("%c", character);
        } else if (strcmp(method, "printf-flush") == 0) {
            printf("%c", character);
            fflush(stdout);
        }

        if (sleep_us > 0) {
            usleep(sleep_us);
        }

        fflush(stdout);
        exit(EXIT_SUCCESS);
    }
}

```

The function receives the character to print, the output method and the waiting time after each output. The signal handling allows the children to stop cleanly when the parent sends `SIGTERM`.

Three output methods were tested:

- `write`: direct low-level output.
- `printf`: buffered stream output.
- `printf-flush`: `printf()` with immediate flushing.

The optional waiting time is implemented here:

```

if (sleep_us > 0) {
    usleep(sleep_us);
}

```

For a 1 ms waiting period, the value 1000 is used because `usleep()` expects microseconds.

3.3 Test 1: Direct Output with write()

The first test was executed with:

```
./scheduling write 0 15
```

The observed output was:

AAABA

The output switched back and forth quickly. Because `write()` bypasses buffers, it shows exactly how the OS switches between the two processes in real-time. Since `write()` writes directly, the output is a good representation of the actual process switching.

In this run, the scheduling appeared quite balanced. However, the exact output can vary between runs because process scheduling depends on the current system load and timing.

3.4 Test 2: Output with Buffered printf()

The second test was executed with:

```
./scheduling printf 0 15
```

The observed output was a large block of A followed by a large block of B:

[illegible]

The output came out in massive blocks. This result was clearly different from the `write()` test because of buffering. `printf()` is a stream function that holds data in memory and only dumps it to the screen when full, which completely hides how the scheduler is actually switching.

Therefore, the visible output is not only caused by scheduling. It is also strongly affected by the buffering behavior of `printf()`.

3.5 Test 3: printf() with fflush()

The third test was executed with:

```
./scheduling printf-flush 0 15
```

The observed output was:

BA
BA
BA

This brought back the alternating pattern. The reason is that after each character, the code calls:

```
fflush(stdout);
```

Forcing the buffer clear makes `printf` look like a direct system call on the screen. However, `fflush()` does not control the scheduler. It only controls when buffered output is displayed.

3.6 Task 2.1a: Changing the Nice Value

For the priority test, the program was started with:

```
./scheduling write 0 30
```

The program prints the PIDs of the two child processes. Then one process can be changed using `renice`, for example:

```
renice +10 -p <PID_OF_PROCESS_A>
```

A higher nice value means a lower priority. Therefore, the reniced process should receive less CPU time, especially when the system is under load.

The command is not part of the C code. The code only prints the PID so that `renice` can be applied from another terminal.

3.7 Task 2.1b: 1 ms Waiting Period

For the waiting-period test, I used:

```
./scheduling write 1000 15
```

The value 1000 means 1000 microseconds, so the process waits for 1 ms after each output.

The observed output was:

```
AABABABABBABABBABAABBAABABABABABABAABABABABBABABA
```

So the answer to task 2.1b is:

Running the program with a 1 ms delay makes the output look very evenly balanced. This happens because each process prints a character and then voluntarily goes to sleep. While it sleeps, it gives up the CPU so the other process can run, reducing direct CPU competition and distributing calculation time more fairly.

3.8 Task 2.1c: Additional CPU Load

To test the effect of an additional time-consuming program, I used:

```
yes > /dev/null
```

This program creates continuous CPU load.

After starting this load, the tests from 2.1a and 2.1b were repeated. Under CPU load, the effect of priorities becomes more visible. When one process is reniced to a lower priority, it receives less CPU time more clearly than before.

For the 1 ms waiting test, the output still remains relatively balanced because both processes voluntarily sleep after each output. However, the system load can make the scheduling behavior less stable.

4 Task 2.2: Concurrent Access to a File

4.1 General Problem

In task 2.2, two processes A and B access the same file. The required sequence is:

```
A writes
B reads
A erases
B writes
A reads
B erases
```

This cycle repeats.

Without synchronization, one process could read while the other process is still writing, or one process could erase the file while the other process is reading. That would be a race condition. Therefore, exclusive access to the file is needed.

5 Task 2.2a: Lock File

5.1 Idea

In `lockfile_access.c`, exclusive access is implemented with a separate lock file.

The files are:

```
#define LOCK_FILE    "shared.lock"
#define DATA_FILE   "shared_data.txt"
#define STATE_FILE   "shared_state.txt"
```

`shared.lock` is the lock file. If it exists, another process is currently inside the critical section. If it does not exist, the file is free.

The important function is:

```
static void acquire_lock_file(void)
{
    while (1) {
        int fd = open(LOCK_FILE, O_CREAT | O_EXCL | O_WRONLY, 0644);
        if (fd >= 0) {
            dprintf(fd, "locked by PID %d\n", getpid());
            close(fd);
            return;
        }

        if (errno != EEXIST) {
            perror("open lock file");
            exit(EXIT_FAILURE);
        }

        usleep(10000);
    }
}
```

The important part is:

```
open(LOCK_FILE, O_CREAT | O_EXCL | O_WRONLY, 0644);
```

Using `O_CREAT` and `O_EXCL` together makes the file creation atomic (all-in-one step). The OS guarantees that if the file already exists, the `open()` call fails instantly. This prevents a race condition because only one process can successfully create the lock file.

The lock is released by deleting the lock file:

```
unlink(LOCK_FILE);
```

5.2 State File

The lock file only protects access. It does not know whose turn it is. Therefore, the program also uses a state file:

```
shared_state.txt
```

It stores the current phase and cycle. The phases are:

```
enum Phase {
    A_WRITE = 0,
    B_READ  = 1,
    A_ERASE = 2,
    B_WRITE = 3,
    A_READ  = 4,
    B_ERASE = 5
};
```

This forces the order:

```
A_WRITE -> B_READ -> A_ERASE -> B_WRITE -> A_READ -> B_ERASE
```

5.3 Observed Output

Running:

```
./lockfile_access
```

produced the expected order:

```
Process A handles phase A_WRITE, cycle 1
Process B handles phase B_READ, cycle 1
Process B reads: Text written by process A in round 1.
Process A handles phase A_ERASE, cycle 1
Process A erased the file.
Process B handles phase B_WRITE, cycle 1
Process A handles phase A_READ, cycle 1
Process A reads: Text written by process B in round 1.
Process B handles phase B_ERASE, cycle 1
Process B erased the file.
```

The sequence is correct, so the lock-file method prevents simultaneous file access.

One thing we noticed during testing is that the lock file is usually not visible with `ls`, because it is created and deleted very quickly. To see it clearly, a delay can be added inside the critical section.

5.4 Advantages and Disadvantages

The lock-file method is simple and easy to understand. It also makes the idea of locking visible, because a real file represents the lock.

The disadvantage is that if the program crashes while holding the lock, the lock file may remain. Then other processes may wait forever until the stale lock file is deleted manually.

6 Task 2.2b: fcntl() Locking

6.1 Idea

In `fcntl_access.c`, the same file-access problem is solved with `fcntl()` instead of a manual lock file.

The files are:

```
#define DATA_FILE  "fcntl_shared_data.txt"
#define STATE_FILE  "fcntl_shared_state.txt"
```

There is no extra lock file. The actual data file is locked directly by the operating system. The lock function is:


```
static void lock_file(int fd)
{
    struct flock lock;

    memset(&lock, 0, sizeof(lock));
    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0;

    if (fcntl(fd, F_SETLKW, &lock) == -1) {
        perror("fcntl lock");
        exit(EXIT_FAILURE);
    }
}
```

The important values are:

- F_WRLCK: exclusive write lock.
- SEEK_SET: start at the beginning of the file.
- l_start = 0: start from byte 0.
- l_len = 0: lock the whole file.
- F_SETLKW: set the lock and wait if it is not available.

Unlocking is done with:

```
lock.l_type = F_UNLCK;
fcntl(fd, F_SETLK, &lock);
```

6.2 Data Access

The `fcntl()` version uses file descriptor functions instead of stream functions.

Writing is done by clearing the file, moving to the beginning and writing the new text:

```
ftruncate(fd, 0);
lseek(fd, 0, SEEK_SET);
write(fd, buffer, (size_t)len);
```

Reading is done by moving to the beginning and reading into a buffer:

```
lseek(fd, 0, SEEK_SET);
n = read(fd, buffer, sizeof(buffer) - 1);
buffer[n] = '\0';
```

Erasing is done with:

```
ftruncate(fd, 0);
```

6.3 Observed Output

Running:

```
./fcntl_access
```

also produced the correct sequence:

```

Process A handles phase A_WRITE, cycle 1
Process B handles phase B_READ, cycle 1
Process B reads: Text written by process A in round 1.
Process A handles phase A_ERASE, cycle 1
Process A erased the file.
Process B handles phase B_WRITE, cycle 1
Process A handles phase A_READ, cycle 1
Process A reads: Text written by process B in round 1.
Process B handles phase B_ERASE, cycle 1
Process B erased the file.

```

This shows that `fcntl()` locking also prevents both processes from accessing the shared file at the same time.

7 Comparison of 2.2a and 2.2b

Both implementations solve the same problem, but they use different locking mechanisms.

Aspect	Lock file	<code>fcntl()</code>
Locking method	Separate file <code>shared.lock</code>	OS file lock
Extra lock file	Yes	No
Main function	<code>open()</code> with <code>O_CREAT</code> <code>O_EXCL</code>	<code>fcntl()</code>
Waiting	Manual loop with <code>usleep()</code>	Automatic wait with <code>F_SETLK</code>
Unlocking	Delete lock file with <code>unlink()</code>	Unlock with <code>F_UNLCK</code>
Main advantage	Simple	Cleaner and OS-supported
Main disadvantage	Stale lock file possible	More complex and advisory

Table 1: Comparison of the two locking methods

The lock-file method is easier to understand, but it can leave a stale lock file after a crash. If a process crashes while holding a lock file, the file stays on disk and causes a deadlock. With `fcntl()`, the Linux kernel automatically removes the lock when the process dies. This makes `fcntl()` much safer, even though it is more complex and advisory. Advisory means all cooperating processes must use and respect the lock.

8 Conclusion

The scheduling part showed that two child processes can compete for CPU time. With `write()`, the output was mostly alternating in our test. With normal `printf()`, the output appeared in very large blocks because of buffering. With `printf-flush`, the output became more direct because the buffer was flushed after every character.

Changing the nice value with `renice` lowers the priority of one process. The effect becomes more visible when the system is under CPU load. Adding a 1 ms waiting period after each output makes the output more balanced because each process voluntarily gives up the CPU.

The file-access part showed why locking is necessary when multiple processes use the same file. Both the lock-file method and the `fcntl()` method kept the required order correct. The lock-file method is simple but can leave stale lock files. The `fcntl()` method is cleaner and OS-supported, and avoids deadlocks if a process crashes.

References

- [1] Prof. Dr.-Ing. Holger Gräßner, *OS Lab 2 - WS 2024 Lab Instructions*, HAW Hamburg, 2024.
- [2] AI tools were used for editing, verification, and formatting support.